ChatGPT

Building an iOS Quiz App from Tech RSS Feeds with Lovable.ai, Cursor & SweetPad

Building an iOS app that transforms tech blog RSS feeds into bite-sized quiz cards is now faster thanks to AI-assisted tools. In this guide, we'll use **Lovable.ai**, **Cursor (AI code editor)**, and **SweetPad** alongside Xcode to develop the app step by step. We'll cover everything from scaffolding the project with AI to parsing RSS feeds, generating quiz cards (potentially with AI), adding user authentication, and preparing for App Store release. Both macOS and Windows development setups are discussed, with tips on cross-platform workflows.

1. Scaffold the App with Lovable.ai

Lovable.ai is an AI-powered app builder that can generate a functional app codebase from a natural language description [1]. We'll start by using Lovable to create the project's blueprint:

- **Sign Up and Create a Project:** Log in to Lovable.dev and create a new app project. In the chat prompt, **describe your app idea in detail**. For example: *"Create an iOS app that fetches tech blog RSS feeds and turns each article into a quiz card with a question and answer. Include user login and a way to track user quiz progress."* Lovable's chat-driven interface lets you iteratively refine this description [1].

- **Include Key Features in Prompt:** Mention specifics like *RSS feed URLs*, *parsing articles*, and *using AI to generate quiz questions*. Lovable can integrate with APIs if asked – for instance, you can ask it to use an OpenAI API to summarize content or generate questions [2] [3]. Also request *user authentication* features (Lovable can scaffold auth using Supabase backend integration) [4].

- **Lovable's Output:** Within seconds, Lovable will draft a codebase (typically a **React/Tailwind front-end with a Supabase backend** by default) for a web app matching your description [1]. You'll see a live preview link of the app. For example, it might generate a React UI that lists feed articles and shows quiz Q&A cards, plus Supabase functions to call OpenAI for question generation [3]. It can also create login/sign-up forms connected to Supabase auth if requested [4].

- **Review and Refine:** Use Lovable's chat to make adjustments. You can say "Add a feature to select which tech blogs' RSS feeds to include" or "Make the quiz cards show one question at a time with a flip for the answer." Lovable's AI will modify the code accordingly. Take advantage of Lovable's versioning and *Visual Edit* mode to fine-tune UI components if needed (e.g., adjust colors or text via the visual editor).

- **Note:** Lovable currently specializes in **web app generation**, not native Swift code. Our plan is to leverage Lovable for the heavy lifting in app logic and backend. We will then transition to a native iOS project using that logic. Essentially, Lovable gives us a working prototype (with backend services) that we'll port into Xcode with AI assistance.

2. Export Lovable Project and Set Up Cursor + SweetPad

Once you have a satisfactory Lovable-generated app, the next step is to bring the code into a development environment where you can build a native iOS app. We'll use **Cursor**, an AI-enabled IDE based on VSCode, which works on macOS (and Windows for editing) to integrate with Xcode via SweetPad.

- **Connect to GitHub:** Use Lovable's built-in GitHub integration to export your project's code. In Lovable, go to **Settings -> Connect GitHub**, and follow prompts to push the generated code to a GitHub repository [5]. This will allow you to pull the code locally.

- **Clone into Cursor:** On your development machine, open **Cursor** (download it from cursor.com for Mac or Windows). Clone the GitHub repo into Cursor. The project will contain front-end code (likely a React web app) and possibly serverless functions (Supabase) from Lovable.

- **Install Dependencies:** If the Lovable project is a web app, you can run it locally (e.g., `npm install` then `npm start`) to understand its functionality. However, our goal is to **create a native iOS app**. You might create a new Xcode project (SwiftUI App) to gradually migrate functionality. Keep the Lovable project open in Cursor as reference.

- **Set Up SweetPad (macOS):** SweetPad is a VSCode/Cursor extension that wraps Xcode's build system so you can compile and run iOS apps without leaving Cursor [6]. This requires a Mac (since Xcode's CLI tools only run on macOS). On your Mac with Xcode installed, do the following in Cursor:

- **Install Xcode Command-line Tools:** Ensure Xcode and its command-line tools are installed and accessible (`xcode-select --install` if not done).
- **Install Xcode Build Server:** Open Terminal in Cursor and run `brew install xcode-build-server`. This enables SourceKit LSP for code completion outside Xcode [7].
- **Install Build Output Beautifier:** Run `brew install xcbeautify` to nicely format xcodebuild output in Cursor's terminal [8].
- **(Optional) Install SwiftFormat:** `brew install swiftformat` for automatic Swift code formatting [9].
- **Add VSCode Extensions:** In Cursor's Extensions panel, install *Swift Language Support* for syntax highlighting [10], and install *SweetPad* extension [11].
- **Generate Build Config:** Press `Cmd+Shift+P` and run `Sweetpad: Generate Build Server Config`. This creates a `buildServer.json` at your project root [12] allowing Cursor to interface with Xcode's build system.
- **Build & Run via Cursor:** Use the SweetPad panel or `Cmd+Shift+B` (if you set a shortcut) to build and run the app on a simulator or device [13] [14]. The first build will index the project for code autocompletion and jump-to-definition in Cursor [15]. You can choose the target (e.g. iPhone 15 simulator) and deploy – all from Cursor without opening Xcode GUI [16] [17].

- **Debug in Cursor:** Optionally, attach the debugger by pressing `F5` and selecting the SweetPad configuration to debug as you would in Xcode (breakpoints, call stacks, etc.) [18].

- **Integrate Lovable Code into Xcode Project:** Now, you'll merge the Lovable-generated logic into your native app:

- **UI Implementation:** Recreate the UI in SwiftUI or UIKit. This is where Cursor's AI shines. For example, open your SwiftUI ContentView in Cursor and write a comment or prompt like "// Create a list of articles with title and a button to take a quiz" and let Cursor's AI auto-complete or suggest code. You can also copy portions of Lovable's React components and ask Cursor (via its Chat `Cmd+L` or inline prompt `Cmd+K`) to convert that into SwiftUI code. Developers have found that prompting Cursor to write SwiftUI views and logic is feasible – one user built an entire SwiftUI app by letting Cursor generate code from scratch and fixing errors along the way [19]. Be prepared to iterate: AI might not be perfect, but it can speed up writing boilerplate.
- **Networking & Data Logic:** The Lovable project likely contains logic to fetch RSS feeds and call the OpenAI API (if included). Identify that logic (e.g., a function that fetches feed URL and another that processes text into questions). You can translate these into Swift. For instance, if Lovable created a Supabase Edge Function for generating quiz questions, you might decide to call that via an HTTP request from Swift. Alternatively, you could implement similar logic directly in Swift (perhaps calling the OpenAI API from the app – more on this in Step 3).

- **Reuse Supabase Backend (optional):** If you stick with the Lovable/Supabase backend, keep the endpoints and database as is. The iOS app can use Supabase's Swift SDK or REST endpoints to fetch data and store user info. This way, you benefit from what Lovable set up (user tables, etc.) without re-writing it. For example, Lovable may have configured Supabase for authentication and a table for quiz scores – your Swift code can directly use those via network calls. *Note:* Supabase has an iOS library to ease integration, or you can use standard URLRequests to the Supabase REST endpoints with the provided API keys.

- **Collaboration and Version Control:** Whether you're on Mac or Windows, maintain the code in GitHub for teamwork and backup. If you make changes in Cursor on Windows (edit Swift code without building), push them, then pull on Mac to build. Lovable's GitHub sync allows you to even push further changes back to Lovable's interface if you want to continue using its UI editor or AI on the web [20] (this is optional, but the integration is two-way via the repo).

3. RSS Feed Parsing and Quiz Card Generation

With the project environment ready, implement the core feature: reading RSS feeds and turning articles into quiz cards. We consider two parts here – **fetching & parsing RSS feeds** and **generating quiz Q&A content**.

- **RSS Feed Fetching:** In Swift, you can fetch RSS feeds using **URLSession** to get the XML data from the feed URL. For example, using a simple data task to retrieve the feed URL contents. Once fetched, the XML needs parsing:
- *Option A: Native Parsing:* Use `XMLParser` (Foundation's XML parser) to traverse the RSS XML. This involves implementing the `XMLParserDelegate` methods to capture elements like item title, link, description, etc. This is a bit low-level but doesn't require external libraries [21].

- *Option B: Use a Library:* You can save time by using a Swift package like **FeedKit** or **AlamofireRSSParser**. **FeedKit** is a popular pure Swift library for reading RSS, Atom, or JSON feeds [22]. It can take a feed URL and give you strongly-typed feed and item objects. For example, after adding FeedKit via Swift Package Manager, you might do:

```
import FeedKit
let feedURL = URL(string: "https://example.com/techblog/rss")!
let parser = FeedParser(URL: feedURL)
parser.parseAsync { result in
    if case let .success(feed) = result {
        if let rss = feed.rssFeed {
            for item in rss.items ?? [] {
                print(item.title)
            }
        }
    }
}
```

FeedKit handles all the XML parsing in the background [23] [24], giving you an `RSSFeed` model with properties for each RSS element (title, description, pubDate, etc.) [25] [26]. This greatly simplifies RSS handling.

- Add the chosen library (FeedKit or others) to your Xcode project (SPM or CocoaPods). Using SPM: in Xcode's **File > Add Packages**, search for the FeedKit GitHub URL and add it. Then implement the feed fetch as shown above.

- **Data Model:** Define a model for the content you need. For instance, a struct `Article` with `title`, `content` (or summary), maybe `link` and an array of `QuizQuestion`. Each `QuizQuestion` could have `question` and `answer` text. This model will help pass data to the UI.

- **Quiz Question Generation:** Turning an article into quiz Q&A can be done in a few ways:

- *Basic Approach:* Use heuristics to generate a question. For example, take the article title or a key sentence and form a question. This is fairly limited (and might not truly reflect article content). Given the power of AI, a better approach is to use an NLP service.

- *AI-Powered Approach:* Leverage an AI service (like OpenAI GPT-4 or others) to summarize or create questions from the article text. You have two main ways to do this:

    1. **On-Device API Call:** Call the OpenAI API directly from the iOS app. You'd send the article text to the API and receive a question/answer. *However*, be careful: embedding a secret API key in a mobile app is not secure (anyone can extract it). If you choose this, you might have the user input their own API key, or accept the risk for a prototype. (Apple's guidelines allow calling out to such services, but you must ensure content is appropriate for your age rating.)
    2. **Backend Function (Recommended):** Use a server or cloud function to do the AI call. This is where **Lovable's Supabase integration** pays off. Lovable can generate a Supabase Edge Function that uses an OpenAI API key (stored securely in Supabase) to process text [3]. For example, your Lovable prompt could have created a function like `generateQuiz(articleText)` that returns a JSON with a question and answer. If so, simply call that from your app (Supabase provides a URL endpoint for your Edge Functions). This keeps the API key off the client side and is more secure.

3. **Alternative Backends:** If not using Supabase, you could set up a simple Node.js or Python cloud function (on AWS Lambda, Firebase Cloud Functions, etc.) to do the same. Firebase, for instance, could host an HTTPS function that calls OpenAI. The concept is similar: the app calls your secure endpoint, which in turn calls the AI API.

- **Integrating Quiz Generation:** Whichever route, integrate it so that when an RSS item is fetched:
  - The app retrieves the article full text. (RSS feeds often provide either a summary or full content. If only summary is given, you might need to fetch the article URL separately – that's more complex and may require HTML parsing. To keep it simple, target feeds that provide enough content in RSS `<description>` or `<content:encoded>`.)
  - Pass that text to the quiz generator. In Swift, this could be an asynchronous network call. Show a loading indicator while the question is generated.
  - Receive the question (and answer). For example, the OpenAI completion might return a formatted Q&A, or your Supabase function might return a JSON like `{"question": "...", "answer": "..."}`. Parse that in Swift and create a `QuizQuestion` object.
  - Update the UI: e.g., navigate the user to a quiz card view showing the question and a button to reveal the answer.

- **AI Model Considerations:** If using OpenAI, craft your prompts carefully for consistent output. E.g., *"You are a quiz generator. Read the following article and produce one question and answer that cover a key point. Article: <text>."* Ensure the response is in a predictable format (maybe ask for JSON). This makes parsing easier. Test this outside the app first with a sample article.

- **Local Testing:** Try out the flow with one known RSS feed (like a popular tech blog's feed) and print the results. Ensure that your parsing correctly extracts articles, and the quiz generation yields sensible Q&A. Using Cursor's AI assistance, you can also let it suggest improvements or troubleshoot (Cursor's chat can answer *"Why is my parser not returning items?"* by analyzing your code).

- **Caching/Storage:** As an enhancement, consider storing generated quizzes locally (Core Data or SQLite) so users can revisit them without regenerating every time. But this is optional – the primary loop is fetch -> generate -> display.

4. Implementing User Authentication

User authentication will allow personalized experiences (e.g., saving a user's quiz progress or feed preferences). There are multiple ways to implement sign-in on iOS. We'll cover two common methods: **Sign in with Apple** and **Firebase Authentication**, as well as mention Supabase Auth if you went that route.

- **Sign in with Apple (Native Apple OAuth):** This is a user-friendly option on iOS that lets users log in with their Apple ID (and is required by Apple if you offer other third-party logins). Steps to implement:
- **Apple Developer Setup:** Make sure your App ID is configured for Sign in with Apple in Apple Developer portal (Certificates, Identifiers & Profiles). In Xcode, enable the *Sign In with Apple capability* for your app target under **Signing & Capabilities** [27]. Xcode will update your provisioning profile accordingly.

- **SwiftUI/UIKit UI:** Use the `AuthenticationServices` framework. SwiftUI provides a `SignInWithAppleButton` view which you can place in your login screen UI [28]. This button takes closures for `onRequest` and `onCompletion`.
- **Handle the Authorization Flow:** In the `onRequest` closure, you can request the user's full name and email (if you need them) [29]. On completion, you'll get a result with either an AppleID credential or an error. Cast the credential to `ASAuthorizationAppleIDCredential` to extract the user's unique ID, name, and email [30]. The `user` ID string is the important one – you can use it to identify the user in your database.
- **Create Account Record:** On first sign-in, you may want to create a user record in your app's database (if using Supabase or Firebase, you'd send this info to those services). **Supabase:** It can accept Apple sign-in via OAuth if configured, but a simpler path is using email/password or magic links (since Sign in with Apple will give you an email; however, often it's a private relay email). **Firebase:** If you use Firebase, there is built-in support to link Sign in with Apple to Firebase Auth (via OAuth token exchange) – more on that below.
- **Persist Login State:** Once signed in, store a flag (e.g., in Keychain or UserDefaults) so next time the app knows the user is logged in. Apple Sign-In also provides a *user identifier* that remains constant for that app and Apple ID combination – you can save this to identify the user on subsequent launches.

Apple's documentation provides a detailed walkthrough of implementing the Sign in with Apple button and handling the callbacks [31] [32]. Test this on a real device if possible (the simulator might require an active iCloud account).

- **Firebase Authentication:** Firebase offers a turnkey solution for email/password and social logins (including Apple). If you prefer Firebase:
- **Integrate Firebase SDK:** Add Firebase to your iOS app (via Swift Package Manager or CocoaPods). Enable Email/Password and Apple as sign-in providers in the Firebase Console for your project.
- **Configure Apple in Firebase:** You still need to configure an *Apple Services ID* and associated keys in Apple Developer and feed those into Firebase, because Firebase will use Apple's OAuth under the hood [33] [34]. (Firebase's documentation guides you through registering your app's Bundle ID with Apple and setting up a Services ID and private key for Sign in with Apple.)
- **Use FirebaseUI or Custom UI:** Firebase doesn't provide SwiftUI controls, so you can either use FirebaseUI (a pre-built UI library) or handle the Apple sign-in manually then pass the credential to Firebase. The flow is: use `ASAuthorizationController` (similar to above) to get an Apple ID token, then create a `AuthCredential` for Firebase: `let credential = OAuthProvider.credential(withProviderID:"apple.com", idToken: idTokenString, rawNonce: nonce)` and sign in to Firebase with `Auth.auth().signIn(with: credential)`.
- **Manage Users:** Once signed in, Firebase gives you a user object. You can use Firebase Firestore or Realtime DB to store additional user info (like which feeds they subscribe to, or their quiz scores). This might overlap with Supabase if you already set that up. It's generally simpler to stick to one ecosystem to avoid redundancy.

Note: If you started with Lovable's Supabase backend, using Supabase Auth might be more straightforward than Firebase. Supabase Auth supports email sign-ups and magic links out of the box, and can integrate with Apple and other OAuth providers too. You would configure Apple as an external OAuth provider in

Supabase and let users sign in that way (they'd be redirected to the Apple OAuth page). However, for an iOS-only app, Sign in with Apple directly might suffice.

- **Testing Auth:** Ensure you test the full cycle: create a new test user, log in, log out, etc. If using Apple, test with a *real Apple ID* (Apple provides a testing mechanism for Apple Sign In in sandbox environment if needed). If using Firebase, test with both Apple and a plain email to see that everything works.

- **Security & Compliance:** Using Sign in with Apple means you must adhere to Apple's user privacy guidelines. For example, Apple only gives the user's name/email on the first sign-in. You should save it then because subsequent logins won't provide those again [35]. Also, any third-party auth (Firebase, Supabase) means you need a **Privacy Policy** (discussed in Step 5) and must handle user data securely.

5. Testing and Preparing for App Store Submission

With development done, it's time to test thoroughly and then prepare the app for distribution via TestFlight and the App Store. Here's a checklist to get your app submission-ready:

- **Debug and Profile:** Run the app on various iOS devices (simulators or physical devices) to ensure it works smoothly. Check for memory leaks or crashes when loading feeds or generating quizzes. Since AI content generation can be slow or occasionally fail, handle those cases gracefully (e.g., show errors or retry options).

- **App Store Assets:** Prepare your app name, description, screenshots, and an app icon. Since your app involves reading articles, ensure your screenshots show the feed list and a sample quiz card to convey the concept.

- **Apple Developer Account Setup:** You already have an Apple Developer account. Make sure your app's **Bundle Identifier** matches what you registered in the Developer portal. Create an App ID there with any capabilities your app uses (e.g., Sign in with Apple, Push Notifications, etc. – push isn't in our scope unless you add it). Also ensure you have a Distribution Certificate and Provisioning Profile. Xcode can manage these automatically if your project's *Signing* is set to "Automatically manage signing" with your team selected.

- **Increment Version and Build:** Set your app version (e.g., 1.0) and build number in Xcode. Clean build and Archive the app (Product > Archive). This produces an IPA ready for upload.

- **TestFlight (Beta Testing):** It's wise to upload to TestFlight first. In Xcode's Organizer, after archive, click "Distribute App" → "App Store Connect" → choose "Upload" for TestFlight distribution. This will upload the build to App Store Connect. In App Store Connect, enable your app for TestFlight, create internal tester groups if needed, and invite testers (you can just test it yourself by installing via the TestFlight app). This step ensures your provisioning and signing are correct and that the app passes Apple's initial automatic checks.

- **App Store Connect Listing:** In **App Store Connect**, create a new app record if not already done. Fill out **App Information** (name, category, etc.) and **Pricing** (set it free unless you plan to charge). **Upload screenshots** for all required device sizes (at least 5.5" and 6.5" iPhones, etc.). Provide a detailed description and keywords. Also provide a support URL and privacy policy URL [36] – **Apple now requires a privacy policy** for any app with account creation or login. You should clearly state what user data you collect (e.g., maybe email if using Apple ID, or usage stats) and how it's used. If your app fetches content from third-party blogs, note that it's sourcing public RSS feeds.

- **App Store Compliance Questions:** When you're ready to submit for review, you'll need to answer Apple's compliance questionnaires [37]:

- *Export Compliance:* Does your app use encryption? If you only use standard HTTPS and Apple-provided encryption, you can usually answer "Yes" (it uses encryption) and then "Qualifies for exemption" (common for apps that just use HTTPS).
- *Content Rights:* Does your app include third-party content? In your case, *yes, it includes third-party blog content.* You should attest you have rights or it's licensed. Since RSS feeds are public, you might claim it is publicly available content. It's a good idea to include at least an in-app acknowledgement or attribution to the source blogs (and perhaps their logos) to avoid any issues. In the review *Notes* field, explain that "The app pulls publicly available RSS feeds from tech blogs and generates quiz questions as a form of commentary/education. No full articles are republished, only short snippets/questions." This will help reviewers understand that you're not scraping content illegitimately but providing a transformative use.
- *Advertising Identifier (IDFA):* If you're not showing ads or tracking installs, answer "No" to using the Ad Identifier.

- **Submit for Review:** Select the build you uploaded in your app listing and click **Submit for Review** [38]. Apple will then review your app, a process that typically takes 1-3 days (check App Store Connect for status updates).

- **App Review Considerations:** Ensure your app **adheres to Apple's guidelines**. A few to watch for:

- No obscene or copyrighted material without permission (your quiz content should be safe given tech blogs).
- If using AI-generated content, make sure it's appropriate and accurate to some degree. Misinformation can be a concern, though in this context it's just quiz questions for learning.
- The app should not crash or have obvious bugs – test thoroughly.
- You must provide an option to delete the user account/data if you have account creation (Apple's Data Safety rules). If using Firebase/Supabase, you should implement a way for users to request deletion of their data, and mention it in your privacy policy.

- Since your app is not primarily user-generated content (it's pulling from known sources), you likely won't run into moderation issues. Just be sure the blogs you include are generally safe-for-work content, as that can affect your age rating.

- **Beta Feedback:** If you had people test via TestFlight, use their feedback to fix any issues before final submission. It's much easier to address problems early.

- **App Store Release:** Once approved, you can release the app immediately or schedule a release. Users will then be able to download your app from the App Store!

6. Cross-Platform Development Tips (Using macOS and Windows)

Developing iOS apps traditionally requires a Mac, but you can leverage both macOS and Windows in your workflow with the right setup. Here are some tips for a smooth cross-platform development experience:

- **Use Windows for Ideation and Cloud Tools:** You can access Lovable.ai from any browser, so feel free to brainstorm and prototype on your Windows PC as well. Lovable's cloud editor doesn't depend on your OS. Likewise, you can run Cursor on Windows to edit code (Cursor supports Windows for editing since it's based on VSCode). This is useful if your Windows machine is more powerful or accessible at times – you can edit Swift code, write documentation, or manage your Git repo.

- **But You'll Need a Mac for Building/Running:** iOS apps *must* be compiled with Xcode on macOS. There's no way around Apple's toolchain requirement. SweetPad, which relies on Xcode CLI, only functions on macOS [16]. Therefore, ensure you have access to a Mac for the actual build/test cycle. This could be:

- Your primary development Mac (recommended for ease).
- A secondary Mac or MacBook on the same network – you could push code to GitHub from Windows and pull it on the Mac.
- A cloud Mac service (like MacStadium or AWS Mac instances) – as a last resort, to run Xcode builds remotely. Some CI services also offer Mac build agents where you can run Xcode builds via scripts (e.g., GitHub Actions with a macOS runner).

- In a pinch, virtualization: running macOS in a VM on Windows (technically possible with tools like VMware or VirtualBox with Hackintosh, or QEMU/UTM) [39] [40]. This isn't straightforward or officially supported, and performance may be slow, so it's not ideal for daily development but could be used to compile a final build if needed.

- **Syncing Projects:** Use Git as the single source of truth for your code. When working on Windows, commit and push changes. When you switch to Mac, pull the latest code and vice versa. This prevents any environment-specific issues. It's also a good practice to add a `.gitignore` that excludes Mac-specific or Windows-specific files (like Xcode's DerivedData or Windows thumbs.db, etc.).

- **Testing on Windows:** You obviously cannot run the iOS simulator on Windows. But you can still write unit tests for any pure Swift logic and run them on a CI or Mac later. For UI, you'll rely on the Mac. If your project had a web component (Lovable's output), you can at least test the web part on Windows to verify the logic for feed parsing or question generation in a non-iOS environment (e.g., running the Lovable web preview to see if feed data appears correctly).

- **Cross-Platform Framework Consideration:** Our approach here is native iOS. If in the future you want an Android version, consider frameworks like **React Native** or **Flutter**. Those allow development on Windows and can compile Android directly on Windows and iOS via Mac. In fact, Lovable's competitor Bolt.new uses Expo/React Native for mobile [41]. You could potentially ask

Lovable to generate a React Native app and follow a similar flow. But for now, using Swift with AI assistance (Cursor) gave us a native app optimized for iOS.

- **SweetPad on Windows:** SweetPad does not operate on Windows because it requires Xcode. If you are primarily on Windows, you might choose to do most coding in Cursor (which will still give AI suggestions) and then do the actual building on a Mac without SweetPad (e.g., open the project in Xcode for final testing). Alternatively, use Remote Development extensions: VSCode (and possibly Cursor) can do remote SSH into a Mac and let you build on the Mac from the Windows UI. That setup is advanced but can be done if you have an always-on Mac server.

- **Keep Mac and Windows in Sync:** Ensure any library or dependency you add is installed in both environments (for example, if you use Homebrew to install tools on Mac, it won't exist on Windows, but you might find Windows alternatives as needed for editing). Keep an eye on line endings (git can normalize CRLF vs LF) to avoid any build script issues.

In summary, **plan your heavy iOS-specific tasks for macOS**. Use Windows for complementary tasks (writing docs, using design tools, or leveraging PC-specific software) and for coding when convenient, but always integrate back to the Mac for running and testing. Many developers use a Windows PC alongside a MacBook, for example – it's a workable setup as long as you have good source control and clear division of tasks.

By following this guide, you leverage the strengths of each tool: **Lovable.ai** to rapidly prototype your app's logic and backend in a conversational way [1], **Cursor** to efficiently write and refactor Swift code with AI pair-programming features, and **SweetPad** to streamline building and debugging the app on device [6] – all while navigating the quirks of iOS development (provisioning profiles, App Store rules) with confidence. Good luck with your app, and enjoy turning tech news into a fun quiz experience for your users!

Sources:

- Lovable.ai Documentation and Comparisons – on AI app generation and integrations [1] [3] [4]
- Cursor and SweetPad Usage – setting up an AI-driven iOS dev workflow in VSCode [6] [12]
- RSS Parsing in Swift – using FeedKit and AlamofireRSSParser for feed data [23] [42]
- Apple Sign In Implementation – adding Sign in with Apple in SwiftUI apps [27] [28]
- Firebase Auth with Apple – configuration needed for integrating Sign in with Apple [33] [34]
- App Store Submission Guide – steps for provisioning, TestFlight, and review compliance [43] [37]

---

[1] [41] Lovable.dev vs Bolt.new vs Fine: Comparing AI App Builders
https://www.fine.dev/blog/lovable-vs-bolt-app-builder-comparison

[2] [3] Integration with AI - Lovable Documentation
https://docs.lovable.dev/integrations/ai-integration

[4] Quickstart - Lovable Documentation
https://docs.lovable.dev/user-guides/quickstart

5 20 Transfer Lovable.dev Projects to Cursor AI - via GitHub Integration - Lovable Videos
https://lovable.dev/video/transfer-lovabledev-projects-to-cursor-ai-via-github-integration

6 7 8 9 10 11 12 15 18 How to use VSCode/Cursor for iOS development | by Thomas Ricouard | Medium
https://dimillian.medium.com/how-to-use-cursor-for-ios-development-54b912c23941

13 14 16 17 39 40 Developing iOS Apps in Cursor with Sweetpad (Yes, It Works) - Daniel's Journal
https://danielraffel.me/2025/04/01/developing-ios-apps-in-cursor-with-sweetpad-yes-it-works/

19 I built an iOS app using Cursor with Sweetpad and Xcode : r/Xcode
https://www.reddit.com/r/Xcode/comments/1lgyeep/i_built_an_ios_app_using_cursor_with_sweetpad_and/

21 XMLParser: Working With RSS Feeds In Swift | by TheDev - Medium
https://medium.com/@MedvedevTheDev/xmlparser-working-with-rss-feeds-in-swift-86224fc507dc

22 FeedKit is a Swift library for Reading and Generating RSS ... - GitHub
https://github.com/nmdias/FeedKit

23 24 25 26 FeedKit on CocoaPods.org
https://cocoapods.org/pods/FeedKit

27 28 29 30 31 32 35 Sign in with Apple on a SwiftUI application
https://www.createwithswift.com/sign-in-with-apple-on-a-swiftui-application/

33 34 Authenticate Using Apple | Firebase
https://firebase.google.com/docs/auth/ios/apple

36 37 38 How to Submit Your App to the App Store in 2025
https://www.instabug.com/blog/how-to-submit-app-to-app-store

42 How to implement an RSS Feed Parser in Swift - iOS App Templates
https://iosapptemplates.com/blog/swift-programming/implement-rss-feed-parser-swift

43 How to setup iOS app with Apple developer account and TestFlight from scratch
https://www.velotio.com/engineering-blog/how-to-setup-ios-app-with-apple-developer-account-and-testflight-from-scratch